

Project Proposal:
Does Jacobian Regularization Help More on Stiff Systems?
An Empirical Study of Neural ODE Solver Cost

Athena Zhong
Course: ENM 5220
Instructor: Professor George Ilhwan Park

May 12, 2026

1 Project Question and Goal

This project studies Neural Ordinary Differential Equations (Neural ODEs) from a numerical analysis perspective. The central question is:

Does the solver-cost benefit of Jacobian regularization depend on the stiffness of the underlying dynamics being modeled? Specifically, does regularization reduce the number of function evaluations (NFE) more when the target system is stiff than when it is smooth, and does this efficiency gain come at any cost to predictive performance?

Finlay et al. [2020] show that Jacobian regularization reduces NFE on their tasks, and Kim et al. [2021] study training difficulty on stiff Neural ODEs. Neither paper tests whether the efficiency benefit of regularization is itself a function of target stiffness; this interaction is the open question this project addresses. The goal is not to propose a new architecture, but to produce a controlled study that isolates the regularization $\times$ stiffness interaction using precisely defined metrics and performance-matched comparisons.

2 Context, Motivation, and Theoretical Foundation

2.1 Why Neural ODEs are a Numerical Analysis Problem

A Neural ODE replaces discrete residual updates with a continuous-time dynamical system:

$$\frac{dx(t)}{dt} = f_{\theta}(x(t), t), \quad x(0) = x_0. \quad (1)$$

Here f_{θ} is a neural network defining the vector field, while a numerical ODE solver computes the forward pass by approximately integrating the dynamics from $t = 0$ to $t = T$. Thus, Neural ODEs separate the model into two components:

- the neural network, which defines the dynamics;

- the numerical solver, which determines how the dynamics are simulated.

This perspective immediately introduces classical numerical-analysis concerns. A model may be expressive enough to fit the task while still producing dynamics that are difficult to solve efficiently. In particular, large local Jacobian norm or stiffness can force a solver to take smaller steps or more function evaluations, increasing runtime and potentially reducing robustness.

2.2 Foundational Literature

The original Neural ODE paper by Chen et al. [2018] established the continuous-depth viewpoint and showed that the forward pass of the model can be computed by an ODE solver. This work motivates the interpretation of deep learning layers as discretizations of an underlying differential equation.

The paper by Finlay et al. [2020] is especially important for this project because it argues that unconstrained learned dynamics can become unnecessarily complicated. They introduce Jacobian and kinetic regularization terms that encourage smoother and simpler flows. Their central claim is highly relevant here: simpler dynamics can reduce solver discretizations and wall-clock time without sacrificing task performance.

A further motivation comes from stiff-Neural-ODE literature. Kim et al. [2021] show that Neural ODE training can become difficult on stiff systems, where different modes evolve on widely separated time scales. In such settings, solver cost and training behavior are strongly shaped by the numerical properties of the learned vector field. This directly connects Neural ODE behavior to traditional concepts from ODE courses, especially stiffness and stability restrictions.

2.3 Key Mathematical Concepts

Local linearization and the Jacobian: For a vector field $f_\theta(x, t)$, the Jacobian matrix

$$J_f(x, t) = \frac{\partial f_\theta}{\partial x}(x, t) \quad (2)$$

describes how the vector field changes locally with respect to the state. Large Jacobian norm typically indicates rapidly varying dynamics, stronger local sensitivity, and potentially more expensive integration.

Jacobian regularization: Jacobian regularization is a soft penalty on the Frobenius norm of the local Jacobian, added to the training loss with weight λ_J :

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} + \lambda_J \cdot \mathbb{E}_{(x,t) \sim \mu} [\|J_{f_\theta}(x, t)\|_F^2], \quad (3)$$

where μ is the empirical distribution over states and times sampled along training trajectories. Conceptually, the penalty discourages f_θ from changing rapidly with respect to the state, with three consequences predicted by classical step-size theory. (i) Since $\|J\|_2 \leq \|J\|_F$, the penalty caps the local Lipschitz constant of the learned vector field. (ii) Since $\rho(J)^2 \leq \|J\|_F^2$ where ρ

denotes spectral radius, it bounds the eigenvalue magnitudes of J , which directly relax the stability step-size restriction of explicit Runge–Kutta methods on stiff problems. (iii) Reducing $\|J\|$ also reduces higher-order time derivatives of f_θ , which lowers the local truncation error of a p -th order RK method and lets the adaptive solver take larger steps. The penalty is thus a principled relaxation of “constrain the local Lipschitz constant” rather than an ad-hoc fix: each consequence above is a direct corollary of standard numerical-analysis bounds. The empirical question this project addresses is whether mechanism (ii), which dominates on stiff problems, makes the penalty disproportionately valuable as target stiffness increases.

Stiffness: Stiff systems contain fast and slow components simultaneously. Even when the true solution behaves smoothly at the macro scale, explicit solvers may be forced to take very small steps to maintain stability — the stable step size scales roughly as $C_p/\rho(J)$, where C_p depends on the method order and $\rho(J)$ is the spectral radius of the Jacobian. This is one of the clearest ways in which a machine learning model can become numerically expensive even if it is statistically successful. Implicit time integration (e.g., backward differentiation formulae, implicit Runge–Kutta) is an alternative strategy that sidesteps the stability restriction entirely: an A -stable method is stable for all $h > 0$ on the linear test problem with $\text{Re}(\lambda) < 0$, so step size is governed by accuracy alone, at the cost of a Newton solve and a Jacobian evaluation per step. Jacobian regularization and implicit integration are thus two distinct strategies for managing stiffness — regularization changes the network so an explicit solver can handle it, while implicit integration changes the solver so an unconstrained network can be tolerated — and the present study compares them directly via the ancillary experiment in Section 3.8.

Solver cost: For Neural ODEs, an especially important practical metric is the number of function evaluations (NFE), i.e., how many times the solver calls the neural network f_θ . This gives a natural computational measure of how difficult the learned dynamics are to integrate.

3 Research Plan and Experimental Design

3.1 Overall Experimental Philosophy

The study uses a full $2 \times 2 \times 2$ factorial design across three binary factors: regularization (baseline vs. Jacobian-regularized), solver type (adaptive vs. fixed-step), and task stiffness (smooth vs. mildly stiff). Crossing all three factors yields eight experimental cells, which allows the main effects and interactions to be estimated from the same set of runs rather than requiring separate one-at-a-time sweeps. The model family is kept intentionally simple so that variation in solver behavior can be attributed to the experimental factors rather than to architectural differences.

3.2 Main Hypotheses

- **H0 (Sanity check):** NFE increases with Jacobian norm and stiffness ratio along learned trajectories. This is not a novel claim. Instead, it follows from adaptive step-size theory but it is included as an explicit baseline validation to confirm the experimental setup is measuring what it claims.

- **H1:** The NFE reduction from Jacobian regularization is significantly larger when the target dynamics are stiff than when they are smooth. Formally: $\Delta\text{NFE}_{\text{stiff}} > \Delta\text{NFE}_{\text{smooth}}$, where $\Delta\text{NFE} = \text{NFE}_{\text{baseline}} - \text{NFE}_{\text{reg}}$ at matched validation MSE. This is the interaction hypothesis that neither Finlay et al. [2020] nor Kim et al. [2021] test directly.
- **H2:** At matched task performance, Jacobian regularization reduces the learned stiffness ratio more under stiff target dynamics than under smooth ones. This tests whether regularization is specifically suppressing stiffness-inducing components of the learned vector field, not just smoothing the dynamics uniformly.
- **H3 (Ancillary):** On stiff target dynamics, a baseline (unregularized) Neural ODE evaluated with an implicit solver achieves wall-clock cost comparable to or lower than a Jacobian-regularized Neural ODE evaluated with an explicit adaptive solver, at matched validation MSE. This positions Jacobian regularization and implicit time integration as alternative strategies for managing stiffness; the comparison clarifies the regime in which regularization is the preferred lever. See Section 3.8.

3.3 Tasks / Datasets

I propose using a small number of tasks with increasing numerical difficulty.

3.3.1 Task A: Smooth dynamics - damped harmonic oscillator

The two-dimensional damped oscillator

$$\dot{x}_1 = x_2, \quad \dot{x}_2 = -\omega^2 x_1 - 2\gamma x_2, \quad (4)$$

with $\omega = 1$, $\gamma = 0.1$. This system has a known analytic solution, low Jacobian norm, and stiffness ratio ≈ 1 . It establishes a clean non-stiff baseline with no ambiguity about the ground-truth dynamics.

3.3.2 Task B: Mildly stiff dynamics - Van der Pol oscillator

The Van der Pol oscillator

$$\dot{x}_1 = x_2, \quad \dot{x}_2 = \mu(1 - x_1^2)x_2 - x_1, \quad (5)$$

with stiffness parameter $\mu = 50$ to $\mu = 100$. At large μ the system exhibits widely separated time scales, inducing genuine stiffness. This is a standard benchmark for stiff ODE solvers and gives a well-characterized, controllable stiffness level.

3.3.3 Task C: Strongly stiff benchmark - Robertson kinetics (optional)

If implementation remains manageable, use the Robertson chemical kinetics system, a standard three-dimensional stiff benchmark with stiffness ratios on the order of 10^5 – 10^{11} . This tests whether the regularization-stiffness interaction extends to extreme stiffness, or whether it plateaus or breaks down.

3.4 Controlled Factors

3.4.1 Factor 1: Vector-field regularity

The following model variants will be compared:

- **Baseline Neural ODE:** no additional regularization ($\lambda_J = 0$).
- **Jacobian-regularized Neural ODE:** penalty term on the local Jacobian Frobenius norm.

The training objective is

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} + \lambda_J \mathcal{R}_J, \quad (6)$$

where $\mathcal{R}_J = \frac{1}{N} \sum_i \|J_f(x_i, t_i)\|_F^2$ is evaluated on sampled trajectory points.

Controlling the confound. A naive comparison of baseline vs. regularized models conflates two things: the effect of the penalty itself and the effect of any change in task performance caused by the penalty. To isolate the NFE effect, λ_J will be selected on a held-out validation set via a sweep over $\lambda_J \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$, and only values for which the regularized model achieves validation MSE within 5% of the baseline’s validation MSE will be included in the comparison. This ensures all NFE and runtime comparisons are made at matched predictive performance, not at matched training objective.

3.4.2 Factor 2: Solver type

The following solver configurations will be compared:

- fixed-step Euler or RK4;
- adaptive Dormand–Prince / RK45 (e.g. `dopri5` in `torchdiffeq`).

This allows direct comparison between:

- fixed discretization cost vs adaptive discretization cost;
- lower-order vs higher-order integration;
- numerical behavior of the same learned dynamics across solvers.

The solver comparison helps show how that numerical difficulty manifests under different numerical methods.

3.4.3 Factor 3: Task stiffness

By changing the underlying synthetic system or the stiffness parameter of the data-generating process, I can create controlled settings where the target problem itself becomes numerically harder.

3.5 Evaluation

The evaluation is organized into three categories: task performance, numerical cost, and numerical difficulty indicators. In this project, Jacobian norm is used as a local diagnostic of vector-field complexity, while stiffness is treated as a separate numerical property associated with multiple time scales and stability-limited integration.

3.5.1 Task performance

- trajectory error (MSE) for dynamical-system regression;

3.5.2 Numerical cost

- NFE (number of function evaluations);
- wall-clock runtime per epoch and/or per forward pass.

3.5.3 Numerical difficulty indicators

- average and maximum Jacobian Frobenius norm $\|J_f(x, t)\|_F$ along trajectories;
- stiffness ratio along trajectories, defined as

$$s(x, t) = \frac{|\lambda_{\max}(J_f(x, t))|}{|\lambda_{\min}(J_f(x, t))|}, \quad (7)$$

where $\lambda_{\max}$ and $\lambda_{\min}$ are the largest and smallest eigenvalues of J_f by absolute value; reported as trajectory mean and maximum;

- adaptive solver step-size statistics:
 - mean step size,
 - minimum step size,
 - variance or histogram of step sizes.
- solver-behavior indicators, if accessible:
 - accepted/rejected step counts,
 - solver failure or warning frequency,
 - sensitivity to tolerance settings (`rtol`, `atol`).

3.6 Statistical Methodology

All experiments will be repeated across five independent random seeds, varying model initialization and the train/validation split. All scalar metrics (NFE, runtime, MSE, Jacobian norm) will be reported as **mean $\pm$ standard deviation** across seeds.

For pairwise comparisons between model variants (e.g., baseline vs. Jacobian-regularized under the same task and solver), a two-sided Wilcoxon signed-rank test will be used rather than a t -test, since NFE distributions are discrete and may be non-normal. Statistical significance is set at $\alpha = 0.05$, with Bonferroni correction applied when multiple comparisons are made within a single experiment. Effect sizes (rank-biserial correlation) will be reported alongside p -values to distinguish statistical significance from practical significance.

Plots will display individual seed runs as scatter points with mean and 95% bootstrap confidence intervals overlaid, so that variability is visible and not hidden by averaged summaries.

3.7 Core Experimental Matrix

The design is a full $2 \times 2 \times 2$ factorial across three binary factors: regularization (baseline vs. Jacobian-regularized), solver type (adaptive RK45 vs. fixed-step RK4), and task stiffness (smooth vs. mildly stiff). This yields eight primary settings:

Setting	Dynamics Variant	Solver	Task Type	Comparison Role
E1	Baseline	Adaptive RK45	Smooth	Reference cell
E2	Jacobian-reg.	Adaptive RK45	Smooth	Regularization effect (smooth)
E3	Baseline	Adaptive RK45	Mildly stiff	Stiffness effect
E4	Jacobian-reg.	Adaptive RK45	Mildly stiff	Regularization $\times$ stiffness interaction
E5	Baseline	Fixed RK4	Smooth	Solver effect (smooth)
E6	Baseline	Fixed RK4	Mildly stiff	Solver $\times$ stiffness interaction
E7	Jacobian-reg.	Fixed RK4	Smooth	Regularization $\times$ solver interaction
E8	Jacobian-reg.	Fixed RK4	Mildly stiff	Full three-way cell

The key comparison is the E1–E2 vs. E3–E4 contrast: does the NFE reduction from regularization differ between smooth and stiff tasks? Settings E5–E8 allow the same question to be asked under a fixed-step solver, where the NFE is fixed by construction and the relevant cost metric shifts to prediction error and step-size adequacy. If time permits, add one strongly stiff task (Task C) as an extended cell for each of the eight settings above, and one tolerance sensitivity sweep on E1 and E3.

3.8 Ancillary Experiment: Implicit-Solver Comparison

The 2x2x2 core compares regularization across two *explicit* solvers (adaptive RK45 and fixed RK4). The numerical-analysis framing of Section 2 raises a natural alternative for handling stiffness: switch to an *implicit* solver, which is not constrained by the explicit stability region and can take much larger steps on stiff problems. I therefore add one ancillary cell, A1, on the mildly stiff task (Van der Pol, $\mu = 50$): baseline (unregularized) Neural ODE evaluated with an implicit solver. The implementation uses `implicit_adams` from `torchdiffeq`; if Newton-iteration logging is required, the fallback is `torchode`’s BDF or `diffra`’s Kvaerno5. The same five seeds and tolerance settings as the core matrix are used.

For implicit methods, NFE alone is not a sufficient cost metric, because each outer step involves a Newton iteration whose per-iteration cost is itself proportional to a Jacobian evaluation plus a linear solve. The ancillary cell therefore reports, in addition to the standard cost metrics:

- outer integrator step count;
- cumulative Newton iterations across the forward pass;
- cumulative Jacobian evaluations (since each Newton iteration evaluates J_{f_θ});
- wall-clock time per forward pass, which is the unit of comparison against the explicit-solver cells.

The decisive comparison is E4 vs. A1 at matched validation MSE: does the regularized explicit-adaptive setup, or the unregularized implicit setup, reach the same accuracy at lower wall-clock cost? Three outcomes are possible. (i) If A1 matches or beats E4, implicit integration substitutes for regularization on stiff problems, and the main practical takeaway becomes “regularize the network *or* change the solver”. (ii) If E4 beats A1, regularization remains the preferred lever, likely because the per-step Newton cost of the implicit solver dominates. (iii) If the two are comparable, regularization and implicit integration are complementary tools attacking different parts of the cost. In all three cases the result strictly sharpens the practical interpretation of the core 2x2x2 result by isolating when Jacobian regularization is the right intervention.

3.9 Planned Plots and Tables

The final paper should include a small number of interpretable figures:

- performance vs NFE scatter plot;
- runtime vs Jacobian norm plot;
- step-size histogram for adaptive solver under smooth vs stiff dynamics;
- table summarizing accuracy / MSE, NFE, runtime, and Jacobian diagnostics;
- possibly a trajectory plot comparing smooth and stiff cases.

4 Implementation Plan and Technical Roadmap

4.1 Software

The project will be implemented in Python using PyTorch and `torchdiffeq`. The latter provides differentiable ODE solvers and supports both adaptive and fixed-step methods, which makes it suitable for a solver-comparison study.

4.2 Model Structure

The model will use a simple Neural ODE block:

$$\frac{dx(t)}{dt} = f_{\theta}(x(t), t), \tag{8}$$

where f_{θ} is a small multilayer perceptron (MLP). The goal is to keep the architecture intentionally simple so that the main independent variables remain the numerical properties of the dynamics and the solver behavior.

4.3 Computational Feasibility of Jacobian Computation

Exact Jacobian. For a state of dimension d , computing the full Jacobian via reverse-mode autodiff requires d backward passes, giving $O(d^2)$ cost per evaluation point. For the low-dimensional systems used here ($d \leq 10$), this is feasible at evaluation time. However, computing exact Jacobians

at every solver step during training is expensive because the regularized model cannot use the adjoint sensitivity method: Jacobian regularization requires the Jacobian to be computed during the forward pass, which forces backpropagation through the solver rather than through the adjoint ODE. This increases memory usage (all intermediate solver states must be retained) and per-step cost. The training time overhead relative to the baseline will be profiled and reported explicitly.

Stochastic approximation. If exact Jacobians become prohibitive, the Frobenius norm can be approximated via Hutchinson’s estimator:

$$\|J_f\|_F^2 = \mathbb{E}_{\epsilon \sim \mathcal{N}(0, I)} \left[\left\| J_f^\top \epsilon \right\|^2 \right] \approx \frac{1}{K} \sum_{k=1}^K \left\| J_f^\top \epsilon_k \right\|^2, \quad (9)$$

where each $J_f^\top \epsilon_k$ requires only a single vector-Jacobian product (one backward pass, $O(d)$ cost). This is the approach used by Finlay et al. [2020]. For the diagnostics reported at evaluation time, exact Jacobians will be computed. For the regularization term during training, the stochastic estimate with $K = 1$ or $K = 4$ will be used if exact computation causes unacceptable slowdown.

Budget estimate. With $d = 4\text{--}10$, a two-layer MLP of width 64, and $\text{NFE} \approx 50\text{--}200$ per forward pass, a single training epoch on a smooth task is expected to complete in under one minute on CPU for the baseline and under five minutes with Jacobian regularization (exact). Five seeds across eight settings (E1–E8) gives a total budget of roughly 40 training runs. At five minutes per run this is under four hours, which is feasible. Runs will be timed in Stage 1 to validate this estimate before committing to the full matrix.

4.4 Approximate Technical Workflow

4.4.1 Stage 1: Baseline setup

1. Implement a Neural ODE model with one adaptive solver.
2. Train on one smooth synthetic task.
3. Verify that NFE, runtime, and prediction error can be logged correctly.

4.4.2 Stage 2: Add diagnostics

1. Implement Jacobian-norm estimation on sampled trajectory points.
2. Log step sizes and adaptive solver statistics.
3. Produce a first correlation analysis between Jacobian size and solver effort.

4.4.3 Stage 3: Add regularization

1. Add Jacobian and optionally kinetic regularization.
2. Train baseline vs regularized models.
3. Compare NFE/runtime/performance tradeoffs.

4.4.4 Stage 4: Add stiffness experiments

1. Replace the data-generating process with a mildly stiff system.
2. Repeat the main comparisons.
3. Evaluate whether regularization is more beneficial under stiff dynamics.

4.4.5 Stage 5: Solver comparison

1. Compare at least one adaptive explicit solver, one fixed-step explicit solver, and (for the ancillary cell A1 on the stiff task) one implicit solver.
2. Keep model and data fixed.
3. Measure how solver choice changes NFE, runtime, adaptive step sizes, Newton iterations (implicit), and predictive performance under fixed learned dynamics.

5 Expected Contribution

The primary contribution is an answer to a question that prior work leaves open: *does the efficiency benefit of Jacobian regularization depend on the stiffness of the target dynamics?* Finlay et al. [2020] show that regularization reduces NFE, but test only on tasks where target stiffness is not a controlled variable. Kim et al. [2021] study stiff Neural ODEs but do not test Jacobian regularization as an intervention. This project runs the experiment that sits at the intersection of those two results.

More specifically, the project will:

1. quantify the regularization $\times$ stiffness interaction: whether ΔNFE from regularization differs significantly between smooth and stiff tasks at matched predictive performance;
2. test whether regularization differentially suppresses the learned stiffness ratio on stiff tasks, using a precise spectral definition of stiffness;
3. provide a reproducible factorial experimental design (five seeds, Bonferroni-corrected tests, performance-matched λ selection) that could serve as a template for future Neural ODE comparisons.

Appendix A

A.1 Possible Definition of Jacobian Penalty

A practical Jacobian penalty could be estimated by sampling trajectory points and using

$$\mathcal{R}_J \approx \frac{1}{N} \sum_{i=1}^N \|J_f(x_i, t_i)\|_F^2, \quad (10)$$

or a stochastic approximation if exact Jacobians are too expensive.

A.2 Stiffness Discussion

In the writeup, stiffness can be introduced through the standard intuition that explicit methods are often forced to take very small stable steps when the system contains both fast and slow modes. This ties the project directly to standard numerical-analysis discussion of stiff ODEs.

A.3 Minimal Deliverables

If the full scope becomes too large, the minimal successful project should still include:

- one smooth task;
- one stiff task;
- baseline vs Jacobian-regularized Neural ODE;
- one adaptive solver and one fixed-step solver;
- metrics: prediction error, NFE, runtime, Jacobian norm.

References

- Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems*, 2018.
- Chris Finlay, Jörn-Henrik Jacobsen, Levon Nurbekyan, and Adam M. Oberman. How to train your neural ode: the world of jacobian and kinetic regularization. In *International Conference on Machine Learning*, 2020.
- Suyong Kim, Weiqi Ji, Sili Deng, Yingbo Ma, and Christopher Rackauckas. Stiff neural ordinary differential equations. *Chaos*, 2021.